

Architecture Decision Record: Technology Stack

Project: Enterprise IT Support Ticketing System / Phase 1 Deliverable

1. Frontend — HTML, Tailwind CSS, JavaScript

We chose a standalone frontend built with HTML, Tailwind CSS, and JavaScript rather than a component framework, and avoided any meta-framework (Next.js, Nuxt, Remix) per the project's architectural constraints.

- Ease and familiarity: every team member already has hands-on experience with plain HTML, CSS, and JavaScript, so we spend our time on the ticketing features instead of learning a new framework's conventions.
- Tailwind CSS was chosen specifically for its simplicity: utility classes let us style forms, the dashboard, and status badges directly in markup, without writing and maintaining separate stylesheets or a component library.
- This keeps the client/server boundary explicit — a plain JavaScript client using `fetch()` — which satisfies the strict client/server separation requirement and keeps CORS and auth-header handling isolated in one place.

2. Backend — Python + FastAPI

We chose Python with the FastAPI framework as the standalone, long-running API server, instead of Node/Express.

- Beginner-friendly: Python's syntax is simple to read and write, which matters for a team still learning backend development during this internship.
- FastAPI over Flask: FastAPI is built on ASGI (Asynchronous Server Gateway Interface), giving us native async request handling, whereas Flask is built on WSGI (Web Server Gateway Interface) and is synchronous by default. ASGI lets the API handle concurrent requests (e.g. dashboard polling, file uploads) more efficiently without extra tooling.
- FastAPI's routing model maps directly onto the REST resources we already planned (auth, tickets, comments), so the API contract and endpoint structure defined in this Phase 1 document do not change.
- Ecosystem support: FastAPI ships with built-in request validation (Pydantic), automatic OpenAPI docs, and straightforward middleware for CORS and JWT auth — all of which this project requires.

3. Database — SQLite

We selected SQLite over PostgreSQL for the relational store.

- Easy to pick up: SQLite's syntax is standard SQL, so the team can apply what we already know instead of learning a new database system.
- Zero-configuration: SQLite runs as a single local file with no server process to install or manage, which fits the 'local full-stack web application' requirement of this project.
- It still enforces the relational structure we need — foreign keys, NOT NULL, and CHECK constraints for our enum-style fields (priority, category, status, role).

4. Summary

Layer	Choice	Primary Reason
Frontend	HTML + Tailwind CSS + JavaScript	Team familiarity; Tailwind's simplicity
Backend	Python + FastAPI	Beginner-friendly; ASGI (async) over Flask's WSGI
Database	SQLite	Easy, standard SQL syntax; zero-config local file